

4.12 Prove that RANDOMPERMUTATION(A) is correct, that is, prove that every possible permutation of A is equally likely to be the output. Also show that the algorithm is no longer correct (it no longer produces every permutation with equal probability) if we change the k in line 2 to n.

1. Invariant

After the iteration for given k finishes, the suffix $A[k \dots n]$ is a uniformly random permutation of the elements that end up there. Independent of the relative order of prefix $A[1 \dots k - 1]$

At step k, algorithm fixates onto random element into position k by picking uniform random index j in range 1 to k and swaps it into place.

2. Prove invariant by induction

Best Case $k = n$

- First iteration, pick j uniformly from 1, ..., n and swap $A[n]$ with $A[j]$. So all elements of the array have probability $1/n$ to be in position n.

- Therefore $A[n]$ is uniform among all elements.

Inductive Step:

- After finishing iteration $k + 1$, suffix $A[k + 1 \dots n]$ is uniformly random over all $(n-k)!$ Orders of elements

- At the start of iteration k, the set of elements in positions $1 \dots k$ is fixed set of k remaining elements

- Choose j uniformly from $1 \dots k$ and swap $A[k]$ element with $A[j]$. Now each of the remaining elements has equal probability $1/k$ to be placed into position k.

- Later positions $k + 1 \dots n$ are not accessed anymore as they are not in the range, so stay uniformly random by the inductive hypothesis.

- THEN, after finishing interaction k, suffix $A[k \dots n]$ is uniformly random

3. Conclusion $\rightarrow$ every full permutation has probability $1/n!$

Take any permutation, for algorithm to output

- At $k = n$, must choose unique j that places $[n]$ into position n, which has probability $1/n$

- At $k = n - 1$, must choose unique j that places $[n - 1]$ into position n - 1, which has probability $1/(n-1)$

- This continues... But at $k = 2$, probability is $1/2$

- Multiply the probabilities together $\rightarrow 1/n * 1/(n-1) \dots * 1/1 = 1/n!$

Since there are $n!$ permutations and each has probability $1/n!$, the output is uniform.

Change k in line 2 to n:

Changing the algorithm so iterations choose j $\leftarrow$ RANDOM(n)

- This opens up possibility of swap with positions that are already fixed

- Once $A[k]$ is set in original algorithm, k position is never touched again.

- If always pick $j \in 1, \dots, n$, then later steps can swap an element that was already set, which

means some permutations will have more random choices than others. This breaks the proof

Example: [1,2,3] and have $k = 3$ and $k = 2$ picking $j \in 1,2,3$

- There are 9 equally likely sequences of random choices

Two outcomes are [1, 2, 3] which occurs 2 times $\rightarrow 2/9$ probability and [2, 1, 3] occurs 1 time $\rightarrow 1/9$ probability.

This counterexample shows that there is not uniform probability on a uniform shuffle on 3 elements, therefore the modified algorithm is not uniform.

4.14 Here is a paranoid algorithm to compute the maximum of a set A of n real numbers:

Algorithm PARANOIDMAXIMUM(A)

1. if $\text{card}(A)=1$
2. then return the unique element $x \in A$
3. else Pick a random element x from A .
4. $x \leftarrow \text{PARANOIDMAXIMUM}(A \setminus \{x\})$
5. if $x \leq x$
6. then return x
7. else Now we suspect that x is the maximum, but to be absolutely sure, we compare x with all $\text{card}(A)-1$ other elements of A .
8. return x

What is the worst-case running time of this algorithm? What is the expected running time (with respect to the random choice in line 3)?

1. Worst case running time

For $n > 1$, the algorithm always does one recursive call on size $n - 1$, 1 comparison for the test on line 5, and $n - 1$ extra comparisons in line 7 are possible

Therefore the time is $T(n - 1) + 1 + (n - 1) = T(n - 1) + n$

And $T(1) = 0$

So $T(n) = (k=2, n) \quad k = (n(n+1))/2 - 1 = O(n^2)$ and $O(n^2)$ is the worst-case running time

2. Expected running time with random choice

Probability the paranoid check even happens

- At size n , c is chosen uniformly from the n elements.
- The paranoid check happens iff x is the max element of A
- That has probability $1/n$

Extra work at size n

- At size n outside of the recursive call, always compute 1 comparison in line 5 and compute $(n-1)$ more comparisons with probability $1/n$

Therefore expected added comparisons at size n are $1 + (1/n)(n-1) = 1 + (n-1)/n = 2 - 1/n$

3. Expected time recurrence

$E(n)$ is expected comparison on input size n .

So, $E(n) = E(n-1) + (2 - 1/n)$, and $E(1) = 0$

4. Conclusion

$E(n) = (k=2, n) (2 - 1/k) = 2(n - 1) - (k=2, n) 1/k = 2(n - 1) - (H_n - 1) = 2n - 1 - H_n$

Since $H_n = O(\log n)$,

$E(n) = 2n - 1 - O(\log n) = O(n)$

There are $2n - 1 - H_n$ comparisons, so therefore the time is $O(n)$ complexity.

4.15 A simple polygon P is called star-shaped if it contains a point q such that for any point p in P the line segment pq is contained in P . Give an algorithm whose expected running time is linear to decide whether a simple polygon is star-shaped.

Need to show is simple polygon P is shaped like a star, where a point q exists where $q \in P$ where every $p \in P$, the segment $qp \subseteq P$

- P is star-shaped iff kernel is nonempty

- Kernel is the set of points that see whole polygon, and is intersection of half-planes where it is one per polygon edge

So need to prove that is the intersection of n half-planes empty or not.

1. Convert the polygon into half plane constraints

- Assume polygon vertices $v_1, \dots, v_n$ are give in counterclockwise order. If they are not, then compute orientation and reverse.

For each directed edge $e_i = (v_i \rightarrow v_{i+1})$ and indice mod n

- Polygon interior is to the left of the directed edge. So define the half-plane $h_i =$ points on or to the left of the line through v_i, v_{i+1}

- Using cross product and orientation test, $h_i = \{x \in \mathbb{R}^2 \mid \text{orient}(v_i, v_{i+1}, x) \geq 0\}$

Then the kernel is $\text{Kernel}(P) = \bigcap_{i=1, n} h_i$

So P is star shaped iff $\bigcap_{i=1, n} h_i \neq \emptyset$

2. Decide if intersection is empty using 2D linear programming

Build $H = h_1, \dots, h_n$ from the polygon edges

Run randomized 2D linear programming algorithm on H with any fixed objective direction c . Ex: $(1, 0)$

- If LP is infeasible, then $R = \emptyset$ so the kernel is empty $\rightarrow$ not star-shaped

- If LP returns a feasible/optimal point q , then $q \in R \rightarrow$ kernel is nonempty $\rightarrow$ then it is star-shaped and output q

Theorem states that 2D LP with n constraints is solvable in $O(n)$ expected time and $O(n)$ storage.

Therefore $O(n)$ is the expected linear time

3. Proof

If the LP finds a point $q \in \bigcap_i h_i$, then q lies in the inside of every half-plane of every edge, so q can see across all polygon edges. This is the kernel condition, so the polygon is star-shaped.

If the polygon is star-shaped, there exists a kernel point q that satisfied all half-plane constraints, so $\bigcap_i h_i \neq \emptyset$, so LP is feasible and will not report as infeasible

Therefore, if it is LP feasible then it is kernel nonempty and polygon is star-shaped

Building the n half planes from the polygon edges = $O(n)$

Running randomized 2D LP on n half-plane constraints = $O(n)$ expected

Therefore, total expected time is $O(n)$

Algorithm:

1. Ensure polygon vertices are CCW
2. For each edge (v_i, v_{i+1}) create half-plane h_i that is left of edge
3. Run randomized 2D LP feasibility and optimization on $H = h_i$
4. If infeasible then return that polygon is not star shaped. Otherwise, return that the polygon is star-shaped

4.16 On n parallel railway tracks n trains are going with constant speeds $v_1, v_2, \dots, v_n$. At time $t = 0$ the trains are at positions $k_1, k_2, \dots, k_n$. Give an $O(n \log n)$ algorithm that detects all trains that at some moment in time are leading. To this end, use the algorithm for computing the intersection of half-planes.

1. Rewrite the motion as lines in (t, y) plane

Train i has position $y_i(t) = k_i + v_i t$

- So each train corresponds to line l_i , so $y = v_i t + k_i$ in the (t, y) plane

A train i is leading at time t iff

- $y_i(t) \geq y_j(t)$ for all j

- $l_i(t)$ is on the upper envelope of the set of lines for some $t \geq 0$

2. Turn the upper envelope into a half plane intersection

For each train i , define the half-plane above its line

- $h_i := (t, y) \mid y \geq v_i t + k_i$

Then the intersection is $C := \bigcap_{i=1, n}$

- This is region of points (t, y) that lie above all lines.

Lower boundary is $y = \max_i (v_i t + k_i)$

Only need times $t \geq 0$, so add the half plane

- $h(t \geq 0) := (t, y) \mid t \geq 0$

Train i is leading at some time $t \geq 0$ iff the line l_i supports edge of boundary of C that lies on lower boundary

So compute C , then read off which constraints h_i become edges on lower chain

3. Make intersection bounded so half-plane algorithm returns traversible polygon

Possibly unbounded intersections, but for extraction need to clip to a box that does not cut the lower envelope.

Bound time to finite interval $[0, T]$ that contains all lead changes

- Find $v_{\max} = \max_i v_i$
- Let $S = \{i : v_i = v_{\max}\}$. Only the ones with maximum k_i can ever be on top for large t
- Pick any $i^* \in S$ with largest k_i . For any train j with $v_j < v_{\max}$, the time when i^* catches j is $t_j = (k_j - k_{i^*}) / (v_{\max} - v_j)$
- Only positive t_j matter, so let $T = \max(0, \max t_j)$
- After time T , train i is guaranteed to be leading for the rest of time, so no new leaders appear after T
- Now choose safe vertical bound Y that stays above envelope on $[0, T]$
- $Y = 1 + \max(k_i + v_i T)$
- Add half planes $t \leq T$ and $y \leq Y$. Do not change which trains are leaders on $[0, T]$ but rather cap the region from increasing

Therefore the final set of half planes are

- $y \geq v_i t + k_i$ for $i = 1 \dots n$
- $t \geq 0, t \leq T, y \leq Y$

Intersection is a convex polygon with $\leq n + 3$ edges

4. Run the half-plane intersection and extract leaders

Use the `IntersectHalfplane(H)` algorithm with running time $O(n \log n)$

Extraction:

Traverse boundary of the resulting convex polygon C and every edge of C lies on the boundary line of one of the input half-planes

- Ignore the edges coming from the caps $t \geq 0, t \leq T, y \leq Y$
 - For each remaining boundary edge, it is on $y = v_i t + k_i$ for some train i . Then add i to the set
- These are the train on top for some time t , and $t \geq 0$

5. Conclusion

By construction for any time t , the smallest y such that $(t, y) \in \bigcap_i h_i$ is $y = \max_i (v_i t + k_i)$

- So lower boundary of the intersection encodes the current leader position

If train i is leading during some time t , then the local maximum is achieved by i , so point $(t, v_i t + k_i)$ lies on boundary and $y = v_i t + k_i$ is satisfied

- h_i is tight and contributes a boundary edge, and report i

If h_i contributes edge on lower boundary, then for all t along the edge, $v_i t + k_i$ is equivalent to the

maximum of the lines, so train i is leading at those times t

Runtime:

Computing $v_{\max}$ and choosing i^* and computing T and $Y \rightarrow O(n)$

Half plane intersection on $n + O(1)$ half-planes $\rightarrow O(n \log n)$

Traversing the boundary and collecting edges $\rightarrow O(n)$

$O(n \log n) + O(n) + O(n) = O(n \log n)$ total time